

Reducing tail latency for multi-bottleneck in datacenter networks: A compound approach

Yuxiang Zhang^a, Lin Cui^{a,*}, Fung Po Tso^b, Xiaolin Lei^{a,*}

^a Department of Computer Science, Jinan University, Guangzhou, China

^b Department of Computer Science, Loughborough University, UK

ARTICLE INFO

Keywords:

ECN
RTT
Congestion control
Datacenter networks
Transport

ABSTRACT

The effectiveness of network congestion control fundamentally depends on the accuracy and granularity of congestion feedback. In datacenter networks, precise feedback is essential for achieving high performance. Most existing approaches use either Explicit Congestion Notification (ECN) or network delay (e.g., RTT) independently as congestion indicators. However, in multi-bottleneck networks, the limitations of these signals become more pronounced: ECN struggles with large cumulative end-to-end latency, while RTT lacks the precision needed to control queuing delays at individual hops. To address these challenges, we propose *Cocktail*, a simple yet effective transport protocol for datacenter networks that combines both ECN and RTT congestion signals to more effectively handle multi-bottleneck scenarios. By leveraging the ECN signal, *Cocktail* bounds per-hop queue lengths, enhancing its ability to control single-hop latency and prevent packet loss. Additionally, by estimating RTT, *Cocktail* effectively manages end-to-end delay, resulting in lower Flow Completion Time (FCT). Extensive experimental evaluations in Mininet demonstrate that *Cocktail* significantly reduces the average and 99th-percentile completion times for small flows by up to 20% and 29%, respectively, compared to current practices under production workloads.

1. Introduction

Datacenter networks run tightly coupled computing tasks that must be responsive to clients. This means up to thousands of backend servers need to exchange information just to serve a single request, and all of the transfers must be completed quickly enough to meet strict deadline [1–3]. To meet these requirements, congestion control mechanisms must simultaneously deliver high network throughput and utilization at low latency. However, when there are multiple network bottlenecks (multi-bottleneck), fulfilling these contradictory demands becomes much more difficult.

Multi-bottleneck is common in today's datacenters as their multi-root tree typologies with oversubscribed aggregation layer link capacities which are prone to creating congestion hotspots [4,5]. Such multi-bottleneck constrains overall performance since it creates congestion hotspots and significantly increases transfer latency. Moreover, it is challenging for end servers to detect the occurrence of multi-bottleneck due to lacking comprehensive congestion signals. Most current schemes rely on a single congestion signal, such as ECN or RTT, leading to performance degradation in multi-bottleneck scenarios.

The main drawback of ECN-based schemes is that they use a fixed base RTT value to calculate the marking threshold, without accounting

for RTT variations. However, since queuing delay constitutes the largest part of RTT, significant RTT variations are common in multi-bottleneck scenarios with varying numbers of bottlenecks. This means the end-to-end latency can increase rapidly as the number of bottlenecks increases, attributed to accumulative single bottleneck latency (refer to Fig. 2).

Moreover, latency-based schemes, whether they use RTT or one-way delay, focus solely on end-to-end delay while neglecting the latency of individual hops. This oversight can lead to performance degradation in multi-bottleneck scenarios because the latency threshold is preset based on multi-hop conditions. For example, DX [6] sets a fixed threshold (headroom) for computing the congestion window using queue latency, operating under the assumption that overall latency is roughly the sum of delays at each switch along the transmission path. However, this mechanism does not account for whether queue latency arises from a single switch or multiple hops. Given the dynamic nature of traffic loads, the number of bottlenecks in a multi-bottleneck scenario can change; for instance, it might shift from five bottlenecks to a single one in the next second. Consequently, the total path delay may decrease while latency at a single congested switch increases. In such cases, long queues or even packet loss may occur at this congested switch, negatively impacting overall network performance (refer to Fig. 3).

* Corresponding authors.

E-mail addresses: samuelzyx0924@gmail.com (Y. Zhang), tcuilin@jnu.edu.cn (L. Cui), p.tso@lboro.ac.uk (F.P. Tso), leixl@jnu.edu.cn (X. Lei).

Motivating experiments are conducted in Section 2, which show that these two signals have their own deficiencies, negatively affecting overall performance in multi-bottleneck scenarios. The results demonstrate that ECN-based schemes cannot well control accumulative latency when the number of bottlenecks goes up. For example, the RTT of an ECN-based scheme can be about 25% higher than a Delay-based scheme due to queuing delay buildup (refer to Section 2). Conversely, relying solely on delay as a congestion signal fails to achieve acceptable queue lengths and loss ratios on individual switches in multi-bottleneck scenarios.

Several recent proposals combine ECN and delay signals to enhance performance. However, how to fully leverage the complementary strengths of the two signals to address the compounded performance issues in multi-bottleneck scenarios still needs to be investigated. For instance, EAR [5] and ECN# [7] share similar design guidelines, which adopt two thresholds in switches for ECN marking. When queue length or single hop sojourn time exceeds the respective threshold, ECN marking is triggered. Although these two schemes incorporate delay into ECN marking, they fail to solve multi-bottleneck accumulative latency problem, where end-to-end latency becomes unacceptable as the number of bottlenecks goes up. FFC [8] is another scheme which uses two signals to adjust the congestion window, expanding it if no ECN marks are detected and reducing it based on the estimated RTT otherwise. Therefore, FFC uses ECN solely to detect congestion rather than combining ECN and delay signals. As a result, it might encounter the accumulative latency problem caused by multi-bottleneck especially when queues of all switches are below the marking threshold.

Motivated by the above challenges, in this paper, we strive to provide a solution which can effectively improve end-to-end latency and mitigate performance degradation on a single switch with the presence of multi-bottleneck. To this end, we present *Cocktail*, a simple yet effective solution to achieve this goal. *Cocktail* detects network congestion by monitoring ECN marks and RTT variations. By leveraging ECN signals, *Cocktail* alleviates performance degradation due to high queue lengths on single hops. Meanwhile, to eliminate cumulative latency caused by multiple bottlenecks, *Cocktail* adjusts its congestion window based on RTT estimations, particularly decreasing the congestion window when an increase in RTT is detected. This is a conservative operation in order to provide low latency delivery.

In summary, the major contributions of this paper are threefold:

- We conducted extensive studies to show both RTT and ECN lack effectiveness and may degrade performance with multi-bottlenecks. Notably, ECN and RTT have their own disadvantages but their advantages are complementary.
- We present a new congestion control mechanism, *Cocktail*, which combines both ECN and RTT. *Cocktail* uses ECN signal to sense the congestion in a single hop and leverage the RTT signal to maximize the network utility and minimize the end-to-end delay.
- We evaluate the performance of *Cocktail* by using Mininet simulations. The results indicate that *Cocktail* can reduce the flow completion time by an average of up to 26.9% compared to state-of-art schemes.

The remainder of this paper is organized as follows. Section 2 introduces the background and related works as well as the motivation for proposing a combining ECN and RTT scheme. Section 3 presents our solution, *Cocktail*, which leverages the strengths of both ECN and RTT signals. Extensive experiments results are shown in Section 5 and demonstrate that *Cocktail*'s effectiveness. Section 7 concludes the paper and discusses potential directions for future work.

2. Background and motivation

2.1. Background

This section will provide a brief introduction to multi-bottleneck and introduce schemes based on different congestion signals.

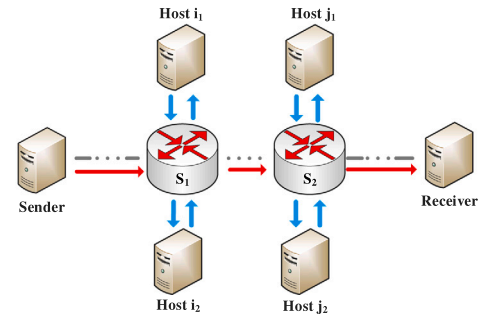


Fig. 1. Multi-bottleneck scenario.

2.1.1. Multi-bottleneck

A flow may traverse many hops to arrive at its destination as shown in Fig. 1. A bottleneck can be formed when a certain switch on flow's transmission path is fully utilized like switches S_1 and S_2 in Fig. 1. This bottleneck throttles the performance of the flow. When a flow encounters multiple bottlenecks along its path rather than just one, this scenario is called a multi-bottleneck. In datacenter, multi-bottleneck is very common due to the high volume transmission. Each switch may fully utilize its capacity to transmit packets, inevitably forming packet queues to handle a large amount of traffic traversing the switch. Consequently, many switches may experience this situation, resulting in flows being delivered under multi-bottleneck conditions.

In datacenters, end-to-end latency is predominantly influenced by queuing delays due to their low-latency fabric design. This means that multi-bottlenecks can significantly impact latency performance because multiple switches with long queues may exist along transmission paths. Additionally, detecting such multi-bottleneck scenarios is challenging for congestion control mechanisms. Currently, most schemes adopt either ECN or delay as the congestion signal which may only reflect if there is congestion occurring. However, it is difficult for senders to determine whether multiple bottlenecks exist on the transmission paths. So it is crucial to investigate the issue of multi-bottleneck and develop effective strategies to solve it.

2.1.2. ECN-based schemes

Significant progress has been made in developing ECN-based transport mechanisms for datacenter networks [1,9–11]. Most of these approaches rely on a similar structure comprising two components: ECN-aware rate control at the end host and ECN marking at the switch [1]. Specifically, an ECN marking threshold K (i.e., standard ECN marking threshold) is given in a switch. When the buffer occupancy at a switch exceeds K , an ECN mark is applied to passing packets. At the end hosts, the sending rate for each connection is adjusted based on the proportion of ECN-marked packets, allowing the buffer occupancy to remain near the marking threshold without sacrificing throughput.

In particular, ECN-based schemes aim to constrain the maximum queue length at a single hop to around the marking threshold of K packets. As a result, flows may experience a latency equivalent to K packets when traversing a fully utilized switch.¹ However, in multi-bottleneck scenarios, the total queuing delay can cumulatively increase with the number of bottlenecks. For instance, the whole queuing delay could reach nK when a flow is traversing n bottlenecks path, which violates the low latency requirement for datacenter network transmission.

¹ For the ease and generality of description, we use the number of packets as queuing delay, i.e., representing the latency by the quantity of packets traversing a switch, rather than specific time interval [1].

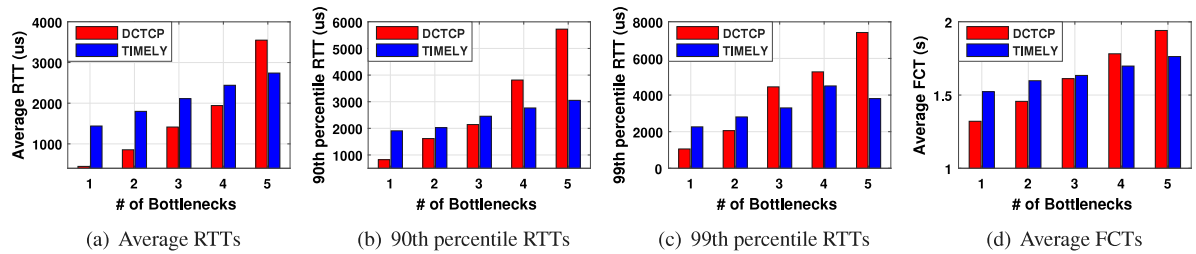


Fig. 2. End-to-end RTTs and FCTs in multi-bottleneck scenarios.

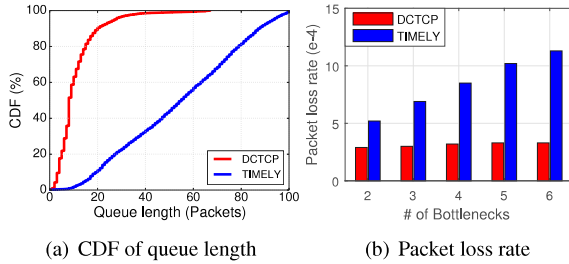


Fig. 3. The comparison of sender switch queuing and loss rate in multi-bottleneck scenarios.

2.1.3. Delay-based schemes

Delay based transports use latency measured at the hosts for sending rate control without touching switches in the network. Recent advances in NIC hardware enable RTT measurements with microsecond-level accuracy, which is sufficient to estimate switch queuing in datacenter networks. Motivated by this observation, several delay-based proposals have been proposed which treat the delay measured at end hosts as the congestion signal, like TIMELY [12] using RTT and DX [6] using one-way delay.

Different from ECN-based schemes, these two mechanisms focus on the delay of the transmission path, neglecting the latency control on a single switch. Take DX as an example. DX uses threshold (headroom) to estimate if there is congestion in the networks, so it would increase its congestion window if the measured delay is less than threshold [6]. However, it cannot sense whether the latency is generated from single or multiple switches. If the delay is caused by one switch with a long queue, a flow can easily encounter packet drops, degrading the performance.

For example, considering a path with five switches, if DX can tolerate 5 packets latency at each hop, the headroom can be set as 25 packets. However, the overall latency may come from certain switches (bottleneck). If other hops are being underutilized (i.e., no queues built up), the queuing delay of this switch queue can reach 25 packets. This significantly deviates from the original assumption that each bottleneck would experience up to 5 packets of latency. Furthermore, this phenomenon can be more severe in larger networks since the headroom might be set higher, potentially leading to packet loss at this single bottleneck.

2.2. Problem exploration

To comprehensively validate the above analysis, we conducted a set of Mininet-based experiments to explore the deficiencies of schemes based on both signals. A multi-bottleneck topology is emulated as depicted in Fig. 1. In this experiment, each link is configured with 1 Gbps link capacity and 25 μ s latency. The marking threshold for DCTCP is set to 20 packets [1]. Besides, each bottleneck (switch) is saturated by 4 long-lived background flows which use the same congestion control as test scheme. In addition, DCTCP and TIMELY are used as the ECN-based and delay-based schemes for comparison, respectively.

Table 1

Path conditions using ECN and RTT.

ECN	RTT	Possible cause	Operation
Exists	Increase	Heavy congested	Shrink
Exists	Decrease	Single hop congested	Shrink
Not exists	Increase	End-to-End congested	Shrink
Not exists	Decrease	Underutilized/No congestion	Expand

End-to-end latency: Although delay-based schemes excel at controlling end-to-end latency, ECN-based schemes often struggle in this regard. To illustrate this, we measured the RTTs and FCTs achieved by DCTCP and TIMELY, and the results are presented in Fig. 2.

The results indicate that as the number of bottlenecks increases, the RTTs of both schemes also increase because flows traverse more bottlenecks and potentially encounter more congestion points and queuing (see Figs. 2(a) ~ 2(c)). Specifically, the RTTs of DCTCP increase more rapidly than that of TIMELY, though DCTCP gets better latency when the number of bottlenecks is relatively small. As shown in Fig. 2(d), TIMELY achieves lower FCT in the presence of multiple bottlenecks due to its effective control of end-to-end latency. Since TIMELY is sensitive to RTT, it reacts to excessive queuing delay by reducing its sending rate, whereas DCTCP focuses on one hop delay and cannot more effectively constrain end-to-end latency. This aligns with the conclusions presented above.

Observation 1. In multi-bottleneck scenarios, ECN-based schemes have limited ability to control accumulative end-to-end latency.

One hop queuing: ECN-based and delay-based schemes response differently to queuing at a switch. To understand how queuing affects both types of schemes, we investigated the queuing behavior of a single switch.

Fig. 3 illustrates the queuing length at the switch connected to the sender switch and the packet loss rate of flows for both schemes. Clearly, TIMELY can have up to 4X more packet loss as the number of bottlenecks increases, as shown in Fig. 3(b). This trend is further explained by the CDF of a single switch queue shown in Fig. 3(a). Although TIMELY has a lower average queuing delay, the tail is significantly larger than DCTCP, leading to higher packet loss. This indicates that delay-based schemes cannot well control per-hop queuing, leading to substantial packet loss, especially in the multi-bottleneck scenario.

Observation 2. Delay-based schemes are infeasible for tackling long queues or packet loss in multi-bottleneck scenarios with dynamics, e.g., the number of bottlenecks is changing.

Summary: Single congestion signal cannot ensure acceptable end-to-end or single hop delays in multi-bottleneck scenario. Performance deteriorates more as the number of bottlenecks increases. These principles derived from our analysis have been summarized in Table 1.

3. Cocktail design

Cocktail aims at utilizing the information provided by both ECN and RTT. Inspired by models used in DCTCP and TIMELY, *Cocktail* consists

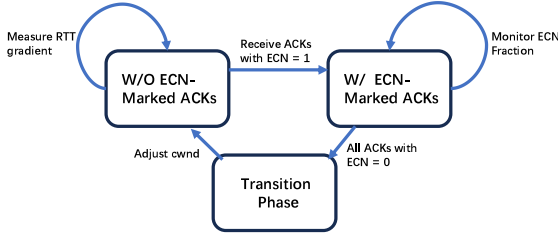
Fig. 4. The state machine of *Cocktail*.

Table 2
Notations in design description.

Notation	Description
W_j	Congestion window at timeslot j
RTT_i	Round trip time at timeslot i
R_i	The sending rate at timeslot i
F_i	The fraction of ECN marked packets at timeslot i
K	The ECN marking threshold at switch
Q_j	The measured queuing delay at timeslot j
λ	A self-updated coefficient parameter
α	A parameter to estimate marked packets
β	A multiplicative decrement factor
σ	The congestion window increase step
RTT_{base}	The minimum RTT observed
q_j	The queue length at timeslot j
C	The link capacity
D	The transmission delay of single packet
N	The number of flows in the network
$flag$	Entry mark for entering transition phase
T_{low}	Filter of RTT spikes

of three parts: retrieving RTT and the mechanism of sending ACKs, congestion condition identification, and the approaches to control sending rate. Fig. 4 shows the design overview of *Cocktail* and notations used in this section are summarized in Table 2.

3.1. Design overview

Based on the analysis in Section 2, *Cocktail* aims to achieve optimal performance by maximizing throughput, minimizing latency, and approaching zero packet loss, even in a multi-bottleneck environment. The core objectives of *Cocktail* are summarized as follows:

- **High throughput:** *Cocktail* should operate with high efficiency, enabling data flows to fully utilize available network bandwidth whenever demand is sufficient.
- **Low latency:** To ensure minimal queuing delays and robust handling of traffic bursts, *Cocktail* must keep buffer occupancy low.
- **Near zero loss:** *Cocktail* should carefully manage queue lengths at switches to prevent packet buildup from exceeding switch capacity, thereby achieving minimal packet loss.

The previous section demonstrates that neither ECN nor delay signals alone can achieve both low packet loss and low RTTs, while they complement each other well. Thus, using both signals as congestion indicators is a promising approach. Therefore, *Cocktail* can use ECN signal to control per-hop queuing and thus packet loss while utilizing RTT signal to control end-to-end queuing and thus bounded packet round trip times.

Moreover, *Cocktail* is a window-based congestion control algorithm which launches a *Fast Start* phase to minimize FCTs of short flows. *Cocktail*'s congestion avoidance follows the popular Additive Increase Multiplicative Decrease (AIMD) rule. Particularly, *Cocktail* uses both ECN and RTT to make a decision on whether to increase or decrease the congestion window in the next round (roughly every RTT). If and only if there are marked packets, the sender reacts by reducing the

Algorithm 1 Cocktail Mechanism

Input: new_rtt, end_frac

Output: new_cwnd

```

1:  $new\_rtt\_diff = new\_rtt - prev\_rtt$ 
2:  $prev\_rtt = new\_rtt$ 
3:  $rtt\_diff = (1 - \alpha) \cdot rtt\_diff + \alpha \cdot new\_rtt\_diff$ 
4:  $rtt\_gradient = rtt\_diff / minRTT$ 
5: if  $ecn\_frac > 0$  then
6:    $flag = 1$ 
7:    $\alpha \leftarrow (1 - g) \times \alpha + g \times F$ 
8:    $cwnd \leftarrow cwnd \times (1 - \alpha/2)$ 
9: else if  $flag == 1$  then
10:   $flag = 0$ 
11:   $Transition\_Phase()$ 
12: else if  $new\_rtt < T_{low}$  then
13:   $cwnd \leftarrow cwnd + \sigma$ 
14: else
15:   $weight \leftarrow W(rtt\_gradient)$ 
16:   $error \leftarrow \frac{new\_rtt - RTT_{ref}}{RTT_{ref}}$ 
17:   $cwnd \leftarrow \theta(1 - weight) + cwnd(1 - \beta \cdot weight \cdot error)$ 
18: end if

```

window by a factor that depends on the fraction of marked packets. On the other hand, if there are no marked packets, the sender adjusts its window based on the RTT gradient. A positive delay gradient due to increasing RTTs indicates rising queues, which means that the sender should decrease the congestion window. On the other hand, a negative gradient indicates receding queues in the network, so the window size should be increased. Other features of TCP such as slow start, fast retransmission when getting three duplicate ACKs or fast recovery from packet loss are left unchanged.

3.2. Cocktail algorithm

The *Cocktail* congestion control mechanism is detailed in Algorithm 1. Guided by the design objectives outlined above, the core principles of *Cocktail* can be summarized as below:

$$W_{j+1} = \begin{cases} W_j + \sigma, & \text{if } g \leq 0, M = 0 \\ W_j \times d_1, & \text{if } g > 0, M = 0 \\ W_j \times d_2, & M = 1 \end{cases} \quad (1)$$

where g is the delay gradient, and M represents ECN marks. α and β are decreasing factors.

There are two conditions: M equals 0 or 1, which means absence and presence of ECN-marked packets respectively. When there is no ECN-marked ACKs, *Cocktail* uses the RTT gradient to update the congestion window. If the gradient is negative or equals zero, *Cocktail* probes for more bandwidth by performing an additive increment for the connection: $W_{j+1} = W_j + \sigma$. When the gradient is positive, the total in-flight is greater than network capacity, *Cocktail* performs a multiplicative window decrement d_1 , which relates to the RTT gradient (which is depicted as W/O ECN-Marked ACKs in Fig. 4). Otherwise, when ECN-marked packets are present, the congestion window is shrunk by d_2 , which relates to the fraction of ECN-marked packets. The well-known AIMD property ensures that our algorithm achieves fairness across connections [12,13] (which is depicted as W/ ECN-Marked ACKs in Fig. 4). The detailed operations of these two conditions will be elaborated below.

With ECN-marked ACK: When the sender receives ECN-marked ACKs, it confirms that the queues at certain switches exceeds the marking threshold. In this case, *Cocktail* decreases the window proportional to the ECN fraction. Specifically, *Cocktail* follows the control law of

DCTCP, where the sender accumulates the ECN fraction as a congestion measurement and adjusts the congestion window accordingly. In particular, the sender maintains an estimation of the fraction of packets that are marked, called α , which is the weighted average of the fraction of marked packets [1,5]. In response to a marked ACK, the window is reduced as $W_{j+1} = W_j \times (1 - \alpha/2)$. This is how *Cocktail* maintains low queue length, while still ensuring high throughput.

Without ECN-marked ACK: If there is no marked ACK in the current congestion window, the sender employs gradient tracking and adjusts the congestion window using a smoothed delay gradient. To compute the delay gradient, *Cocktail* calculates the difference between two consecutive RTT samples, which is shown in line 1 ~ 3 of Algorithm 1. *Cocktail* then normalizes this difference by dividing it by the minimum RTT, obtaining a dimensionless quantity [14]. This approach ensures that all flows have knowledge of the bottleneck's queue length, which provides two key benefits. First, the system can have a unique fixed point determined by the RTT (refer to Section 4). Second, all flows can converge to the same rate, since they share the knowledge of the bottleneck RTT. In practice, the exact value of the minimum RTT does not matter since the sender only needs to determine if the queue is growing or shrinking. Therefore, a fixed value is used to represent the wire propagation delay across the datacenter network, which is known ahead of time. A continuous weighting function $w(g)$ is then designed to make the transition between window increase and decrease smooth, avoiding the on-off behavior that causes oscillation [14]. Here, we simply use a linear function of w_i :

$$g_i = \begin{cases} 0, & g_i \leq -\frac{1}{4} \\ 2g_i + \frac{1}{2}, & -\frac{1}{4} < g_i < \frac{1}{4} \\ 1, & g_i \geq \frac{1}{4} \end{cases} \quad (2)$$

Then, Eq. (1) is transformed to Algorithm 1.

While we believe the above control mechanism is effective in each condition mentioned in Table 1, its performance during transitions between these conditions is uncertain. A special case needs to be considered: If ECN signals exist in W_j while no ECN signals have been received in W_{j+1} , it suggests that the excessive queue lengths in all switches along this path have been eliminated. Since the W_j must be a shrinking operation, the RTT gradient is likely negative in W_{j+1} . Hence, W_{j+1} would equal to $W_j + i$. At this time, if $NW_j = CR$, in W_{j+1} the queue length in certain switches along this path would exceed the marking threshold, resulting in fluctuation. Therefore, a new method is needed to address this reversed congestion window adjustment.

3.3. Transition phase

The transition phase is defined as follows: when the sender receives ECN signals in W_j and gets no ECN signals in W_{j+1} , the computation of W_{j+1} is defined as the transition phase. In this phase, *Cocktail* shrinks the congestion window in order to avoid fluctuation. Inspired by prior work [6,8], the following formula is adopted:

$$W_{j+1} = W_j \times (1 - \frac{Q_j}{\lambda}) \quad (3)$$

where Q_j is the queuing delay at time j and can be approximately computed as $RTT_j - RTT_{base}$.

The decrease should be just enough to prevent fluctuations without compromising utilization. An overly aggressive reduction of the congestion window can lead to under-utilization of network capacity. For *Cocktail*, the exact amount depends on the number of flows sharing the path, as the total in-flight packets should decrease to avoid exceeding the marking threshold. Coefficient λ is used to account for the number of competing flows. The value of λ is derived below.

Assume that at time j , the queue lengths of all bottleneck switches are equal to the marking threshold. Let $W_j^{(k)}$ and Q_j^k denote the congestion window and the queuing delay of flow k at time j , respectively.

Therefore, the sum of the window size which shares the same path equals the bandwidth-delay product $C \cdot RTT_{base}$:

$$\sum_{k=1}^N W_j^{(k)} = C \cdot RTT_{base} \quad (4)$$

Since none of the N flows experiences congestion, they all increase their window by σ at time $j+1$:

$$\sum_{k=1}^N W_{j+1}^{(k)} = C \cdot RTT_{base} + N\sigma \quad (5)$$

At time $j+1$, the combined congestion windows of all flows cause the queue length to exceed the marking threshold. Assume that $Q_{j+1}^{(k)}$ is the part of the queuing delay that the k th flow has experienced due to the excess queue length beyond the marking threshold. Thus, the total number of packets in the network at time $j+2$ can be calculated from the sum of the congestion window of all flows.

$$\sum_{k=1}^N W_{j+2}^{(k)} = \sum_{k=1}^N W_{j+1}^{(k)} \times (1 - \frac{Q_{j+1}^{(k)}}{\lambda}) \quad (6)$$

Assuming every flow experiences maximum queuing delay $N \cdot D$, where D is the transmission delay of a single packet. Then, we get:

$$\begin{aligned} \sum_{k=1}^N W_{j+2}^{(k)} &= \sum_{k=1}^N W_{j+1}^{(k)} \times (1 - \frac{N \cdot D}{\lambda}) \\ &= (C \cdot RTT_{base} + N) \times (1 - \frac{N \cdot D}{\lambda}) \end{aligned} \quad (7)$$

The total number of in-flight packets at time $j+2$ should be equal to the bandwidth delay product:

$$(C \cdot RTT_{base} + N) \times (1 - \frac{N \cdot D}{\lambda}) = C \cdot RTT_{base} \quad (8)$$

Since $D = 1/C$, solving for λ from Eq. (8) results in:

$$\lambda = \frac{N \cdot D}{1 - \frac{C \cdot RTT_{base}}{C \cdot RTT_{base} + N}} \quad (9)$$

Among the variables required to calculate λ , the only unknown is N , which is the number of concurrent flows. The N can be estimated according to the current congestion window since *Cocktail* achieves fair-share at a steady state. Then Eq. (5) can be rewritten as:

$$\begin{aligned} \sum_{k=1}^N W_{j+1}^{(k)} &= N \times W_{j+1}^{(k)} = C \cdot R_{base} + N \\ \Leftrightarrow N &= \frac{C \cdot R_{base}}{W_{j+1}^{(k)} - 1} \end{aligned} \quad (10)$$

Using Eq. (9) and replacing D , single packet transmission delay, with $1/C$, we get:

$$\lambda = \frac{R_{base} \cdot W_{j+1}^{(k)}}{W_{j+1}^{(k)} - 1} \quad (11)$$

To compute λ , the sender only needs to know the base RTT R_{base} , and the previous congestion window. No additional measurement is required. Unlike purely ECN-based solutions, calculating λ does not need to rely on external configuration or parameter settings. Moreover, variations in link capacity across the network do not affect the calculation of λ .

3.4. Fast start

In current datacenter transports, many approaches transmit short messages without regard for potential congestion [15,16]. Hence, in *Cocktail*, new flows initiate by transmitting a sufficient amount of data (i.e., *Fastbyte*) to minimize the FCTs of short messages. The receiver subsequently returns explicit congestion information to manage future transmissions without introducing additional delays. Such blind transmissions mean that buffering can occur when multiple senders transmit to the same receiver [15]. While some buffering is unavoidable to achieve low latency, it paradoxically contributes to increased latency when it occurs. Many previous designs have attempted to reduce buffering [1,6,9,10,12], but none of these approaches can completely

eliminate the latency penalty of buffering. Thus, *Cocktail* adopts the stringent scheme proposed above can help to reduce the buffering then control the transfer latency.

Furthermore, Fast start can be overly aggressive in the case when a large number of flows arrive in a burst. A new flow that initiates a fast start may experience excessive packet loss. Without a quick loss recovery scheme, a new flow may end up with a worse flow completion time than if fast start had not been used at all. To address this, *Cocktail* enables per packet ACK at the receiver to quickly inform the sender which packets have been received to help the sender detect packet loss quicker. Retransmitting lost Fastbytes in a fast start way results in a very slow loss recovery procedure, as retransmitted packets may get lost again in the network. To prevent this, *Cocktail* retransmits these lost packets under congestion window control.

3.5. Parameter settings

RTT low threshold T_{low} . Since bursts may create transient queues in the network, resulting in RTT spikes during occasional collisions. Without careful handling, the sender might interpret an RTT spike as a positive gradient, mistakenly inferring congestion and backing off unnecessarily. To mitigate this issue, *Cocktail* employs a low threshold T_{low} to filter RTT spikes and the RTT gradient is calculated on RTT samples which are larger than T_{low} [6,12].

Fast start threshold *Fastbyte*. Blind transmissions can cause buffering when multiple senders transmit to the same receiver, potentially overwhelming switch capacity and filling up buffers. To mitigate this, *Cocktail* sets a threshold, *Fastbyte*, to limit such occurrences and control overall latency. In current *Cocktail*, the *fastbyte* is set as 10 packets since most short flows can be transferred within such setting, preventing other latency caused by mechanism control [15].

4. Analysis of cocktail

This section presents an analysis of *Cocktail*. First, we prove that *Cocktail* has a unique fixed point determined by RTT. Next, we demonstrate that all *Cocktail* flows can converge to the same rate. Finally, the queue length of a single hop remains below the maximum observed of DCTCP.

Particularly, this section focuses on analyzing the steady behavior of *Cocktail* which relates to the operation of line 11~13 in Algorithm 1. The line 6~7 and line 9 operation in Algorithm 1 is a monotonic function, which means the state machine remains in these operations only briefly. For example, when the sender receives an ECN-marked ACK, it performs the window reduction operation specified in in Algorithm 1 line 6 & 7. The sender then transitions to other operations in Algorithm 1, without reaching stasis in this monotonic decreasing function. The same applies to line 11 of Algorithm 1.

Thus, steady-state behavior can only be achieved through the operations in lines 11~13 of Algorithm 1, as these include both window increase and decrease functions. The operations of these three lines can be formulated as follows:

$$\frac{dW_{j+1}}{dt} = \frac{1 - W_j \delta}{RTT_{base}} - \frac{W_j \beta g_i}{RTT_{base}} \cdot \frac{q(t - RTT_{base}) - q'}{q'} \quad (12)$$

where W_i , the weight of window decreasing, is a function of g_i , and must satisfy $0 \leq W_i(g_i) \leq 1$ for any g_i . For ease of description, we assume the feedback delay of congestion signals as RTT_{base} [14]. Intuitively, $W_i(g_i)$ is monotonically increasing with g_i , because a larger RTT gradient should lead to a larger window decrease. $W_i(g_i)$ is an indicator function of g_i , i.e., $W_i(g_i) = 1$ when $g_i \geq 0$, and $W_i(g_i) = 0$ when $g_i < 0$.

4.1. Fixed point

We first obtain the fixed point of the Algorithm 1.

Theorem 1. *The Cocktail scheme described in Algorithm 1 has a unique fixed point. All flows have the same congestion window at this fixed point, and the queue length is $q^* = \frac{q' \delta}{W_j \beta} + q'$.*

Proof. By setting the left-hand side of Eq. (12) to 0.

$$\frac{dW_{j+1}}{dt} = 0 \quad (13)$$

Then any fixed points of the *Cocktail* (if they exist) must satisfy:

$$\frac{1 - W_j \delta}{RTT_{base}} = \frac{W_j \beta g_i}{RTT_{base}} \cdot \frac{q(t - RTT_{base}) - q'}{q'} \quad (14)$$

At any of the fixed points, let us assume the queue length at the fixed points is q^* and it is shared by all flows and determined by Eq. (14):

$$q^* = \left[\frac{(1 - W_j) \delta}{W_j \beta W_i} + 1 \right] \cdot q' \quad (15)$$

Since it is a fixed point, all flows should be stable and converge to a fair share. Thus, the gradient of all flows equals to 0, which means the $w_i = 0$. Combining with Eq. (15), the equation can be rewritten as:

$$q^* = \frac{q' \delta}{W_j \beta} + q' \quad (16)$$

Thus, *Cocktail* has a unique fixed point which leads to a unique fixed point of queue length q^* . $\square$

4.2. Convergence

In this section, we demonstrate that two *Cocktail* flows with the same RTT can converge to their fair shares.

Theorem 2. *Two Cocktail flows converge to fair share under the network model as shown at the beginning of this section with the same round trip delay.*

Proof. Jain's fairness index is used, which is defined as follows, $F(x) = \frac{(\sum x_i)^2}{n(\sum x_i^2)}$. Besides, as mentioned before, flows would converge to the fixed share of network capacity only when they stay in the steady phase. Otherwise, they may increase or decrease the congestion window, which violates the flows' convergence.

Let us consider two *Cocktail* flows, whose window sizes are x_1 and x_2 , respectively (assuming $x_1 < x_2$). We see $\Delta F \geq 0$, if and only if $\Delta x_1/x_1 \geq \Delta x_2/x_2$. There are three conditions,

- If $g_i \leq 1/4$, then w_i equals to 0. Then $\Delta x_1 = \Delta x_2 = \delta$, since the increment of congestion window is δ . As assumption denotes $x_1 < x_2$, it is easy to see $1/x_1 > 1/x_2$. Thus, the formula becomes $\Delta x_1/x_1 > \Delta x_2/x_2$.
- If $g_i \geq 1/4$, then w_i is set as 1. Thus, the formula can be rewritten as $\Delta x_1/x_1 = -\beta \cdot error = \Delta x_2/x_2$.
- Last, if $g_i \in (-1/4, 1/4)$, then we get $\Delta x_1 = \delta(1/2 - 2g_i) - \beta \cdot error \cdot (2g_i + 1/2) \cdot x_1 > \delta(1/2 - 2g_i) - \beta \cdot error \cdot (2g_i + 1/2) \cdot x_2 = \Delta x_2$. $\therefore \Delta x_1/x_1 > \Delta x_2/x_2$. When the two streams share the same bottlenecks and the propagation delay is the same, the gradient observed by the two streams should also converge to the same value. So the fairness index keeps increasing.

In summary, in each case, the index is strictly non-decreasing. Therefore, eventually, the two *Cocktail* flows converge to their fair share. $\square$

Table 3
Flow size distribution of realistic workload.

	Web server	Cache follower	Web search	Data mining
0–100 KB	81%	53%	52%	83%
100 KB–1 MB	19%	18%	28%	8%
> 1 MB	0%	29%	20%	9%
Average flow size	64 KB	701 KB	1.6 MB	7.41 MB

4.3. Queue length

Next, the worst case of queue length, in which the accumulative queue length arrives at the upper bound.

Theorem 3. *The maximum queue length has an upper bound: $nK + N$.*

Proof. We consider N long-lived flows with identical round-trip times RTT, sharing bottleneck link of capacity C . We assume that the N flows are synchronized, which is an assumption that is only realistic when N is small where is the case *Cocktail* primarily addresses in datacenters. Under these conditions, the expected maximum queue length of single hop Q_{max} equals K .

Hence, the queue lengths of all switches reach K at time T_i and the sum of latency of this path arrives at nK . Worse still, at T_{i+1} , all flows increase their congestion window by i . In this case, the latency arrives at $nK + N$. However, there are two conditions when the congestion window enforces adding i when the overall latency in the network equals nK . First, the previous operation on the congestion window was a decrease, then at T_{i+1} , the congestion window is reduced. This makes it impossible for the worst-case scenario to occur. Another condition is adding i to their congestion window as time T_i . Then, the RTTs of all flows have undoubtedly increased. Consequently, their congestion window would not increase at time T_{i+1} . In summary, the worst case cannot happen and the overall queue length does not reach $nK + N$. $\square$

5. Implementation and evaluation

In this section, we first present some implementation details. Then, Mininet based experiments are conducted to answer the following three key questions:

- **How does *Cocktail* perform in multi-bottleneck scenario?** In a static experiment, the results show that *Cocktail* reduces RTTs, maintains a low loss ratio, and ensures fairness between flows with long and short RTTs.
- **Does *Cocktail* scale to large datacenter fabric?** Using large-scale Mininet simulations, we show that *Cocktail* scales to multi-hop topologies and delivers the best overall performance. For example, it reduces the average FCT for large flows by up to 44.48% compared to TIMELY, while achieving up to 11% lower 99th percentile FCT for small flows compared to the DCTCP. Large-scale Mininet simulations demonstrate that *Cocktail* scales effectively to multi-hop topologies, delivering outstanding overall performance. For instance, it reduces the average Flow Completion Time (FCT) for large flows by up to 24.5% compared to TIMELY and achieves up to an 24.7% reduction in 99th-percentile FCT for small flows compared to DCTCP.
- **How robust is *Cocktail* to parameter settings?** Using a series of targeted simulations, we show that *Cocktail* is robust to *Fastbyte* settings.

5.1. Implementation

We have implemented *Cocktail* on the Linux kernel 4.10 by plugging a new congestion control module.

Cocktail relies on ECN-enabled switches to generate congestion notifications, a feature widely supported in modern datacenters [17,18].

Accurate RTT measurement is also essential for *Cocktail*, requiring servers equipped with high-resolution timers (up to the microsecond level) to support high-speed, low-latency datacenter networks. Fortunately, the option of a high-resolution timer has been provided in the Linux kernel 2.6 and later versions. Besides, one can install a NIC that supports the DPDK driver to use software NIC timestamp or directly equip hardware timestamp to obtain more accurate measurement [6,8,12].

To eliminate host processing delay, we perform TX timestamping just before sending packets to the NIC and RX timestamping immediately after packets are received, with the DPDK-based device driver [6]. We utilize *rdtsc* to capture CPU cycles and convert them into nanosecond precision. The memory overhead is proportional to the arrived data of which the corresponding ACK has not been sent yet. The memory overhead is negligible as it requires storing 8 bytes per in-flight packet.

The majority of *Cocktail*'s congestion control logic is implemented in the function that processes ACK packets. Each ACK packet carries pre-calculated RTT information in its header, provided by the DPDK-based device driver. For each congestion window round, the received delay samples are averaged and used to update the congestion window.

5.2. Experiment setup

Network Topology: We employ a widely used fat-tree topology with 10 pods, in which there are 250 servers, 25 core switches, 50 aggregation switches, and 50 edge switches [8]. Each link is configured with a bandwidth of 40 Gbps and a latency of 10 μ s. The network load is varied from 10% to 90% to evaluate performance across different conditions.

Benchmark Workload: We use four traffic distributions based on measurements from production datacenter networks, as summarized in Table 3 a web search workload [1], a data mining workload [19], a web server workload [20] and a cache follower workload [20]. In general, both workloads are heavy-tailed. Overall, these workloads exhibit heavy-tailed characteristics. For instance, in the web search workload, approximately 60% of all bytes come from flows smaller than 10 MB, resulting in frequent concurrent flows on the same link, which increases network contention. The data mining workload, by contrast, primarily consists of tiny and large flows [15]; around 80% of flows are smaller than 10 KB, while approximately 95% of the data volume is from large flows.

Transports Compared: We evaluate the performance of five transports, *Cocktail*, DCTCP, TIMELY, EAR and HPCC. In addition, ECMP is used for routing and load balancing.

Performance metric: Unless stated otherwise, FCT serves as the primary performance metric. We evaluate both the overall average FCT across all flows and the average FCT for flows of different sizes (small, medium, and large). To assess tail latency, the 99th-percentile FCT for small flows is presented in the figures. All results reflect the average of five runs.

5.3. Static flow experiment

We begin with a basic static flow experiment to evaluate whether *Cocktail* can address the challenges outlined in Section 2. By repeating the experiments from Section 2 with the addition of *Cocktail*, we observe that *Cocktail* effectively tackles multi-bottleneck issues.

Fig. 5(a) shows that *Cocktail* achieves low RTTs comparable to those of DCTCP, TIMELY and EAR. Specially, *Cocktail* can smooth the increment of RTT when the number of hops goes up, indicating its effectiveness in reducing end-to-end latency. Fig. 5(b) demonstrates that *Cocktail* can reduce FCT by an average of 11.2%. Thanks to its efficiency in minimizing RTT, *Cocktail* can outperform DCTCP and TIMELY in FCT performance and shows similar improvements to EAR. Additionally, Fig. 5 confirms that EAR struggles with cumulative latency in multi-bottleneck scenarios, as previously observed.

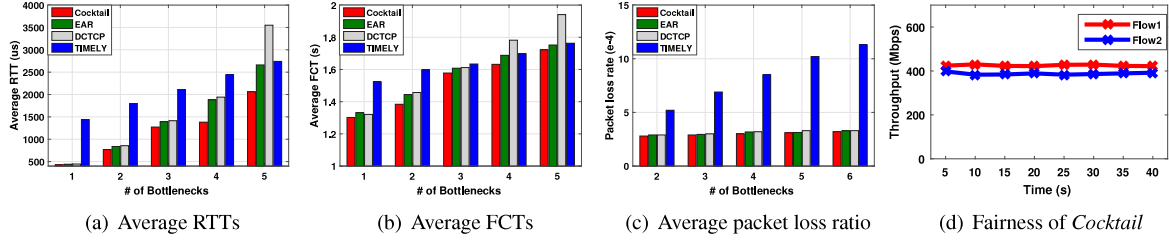


Fig. 5. Static experiments: (a) Average RTTs; (b) Average FCTs; (c) Average packet loss ratio; (d) Fairness of *Cocktail*.

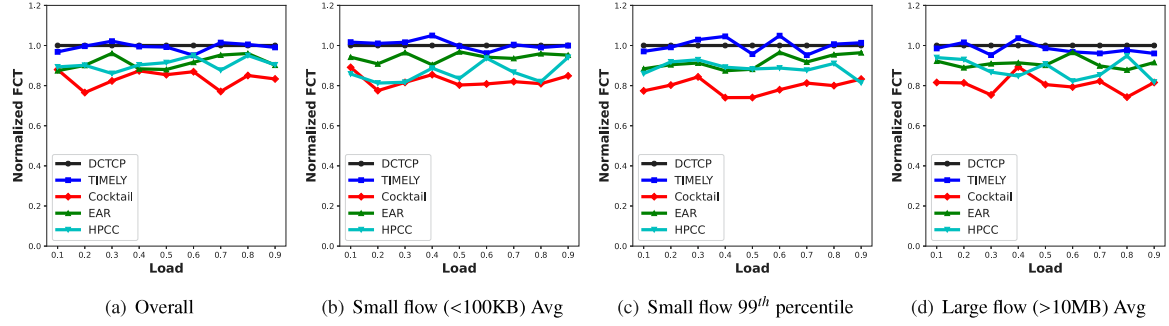


Fig. 6. Web search workload: FCT across different flow sizes.

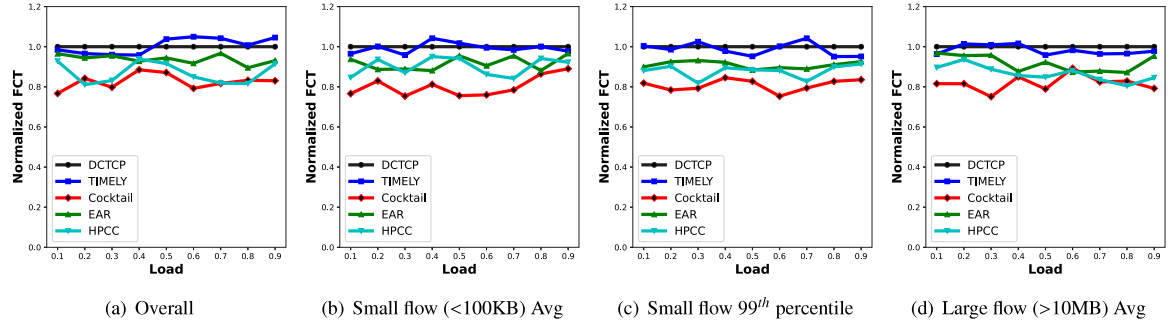


Fig. 7. Data mining workload: FCT across different flow sizes.

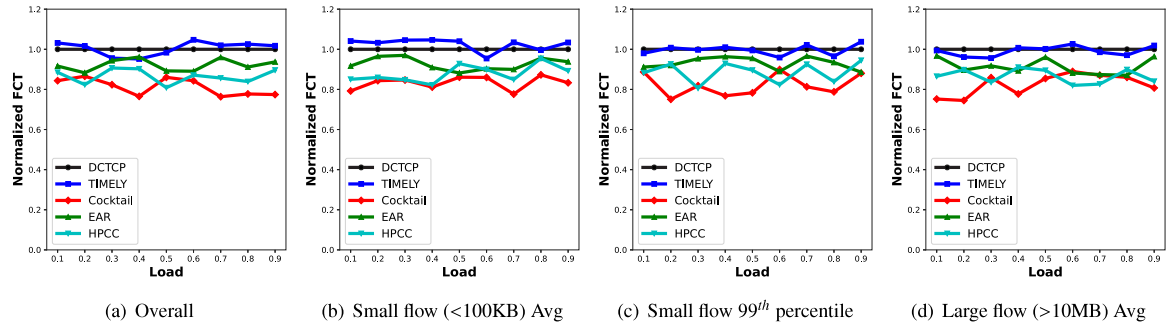


Fig. 8. Hadoop workload: FCT across different flow sizes.

Further, Fig. 5(c) presents the loss ratio achieved by *Cocktail*, which achieves a comparable loss ratio compared to DCTCP and EAR while getting better performance than TIMELY. Lastly, to assess throughput fairness, two *Cocktail* flows were generated to coexist. Fig. 5(d) shows that the throughput difference between two flows is relatively small, which means *Cocktail* can offer fair performance to each *Cocktail* flow.

These results demonstrate that *Cocktail* performs well in multi-bottleneck scenarios and achieves the objectives set out in Section 3.

5.4. Large-scale simulations

In this section, we evaluate the performance of *Cocktail*, DCTCP, TIMELY, EAR and HPCC through large-scale simulations. At the start of each simulation, the flow generator module randomly selects a sender and a receiver to initiate a new flow, with each flow generated using a Poisson process. Figs. 6 to 9 present the FCT results across different

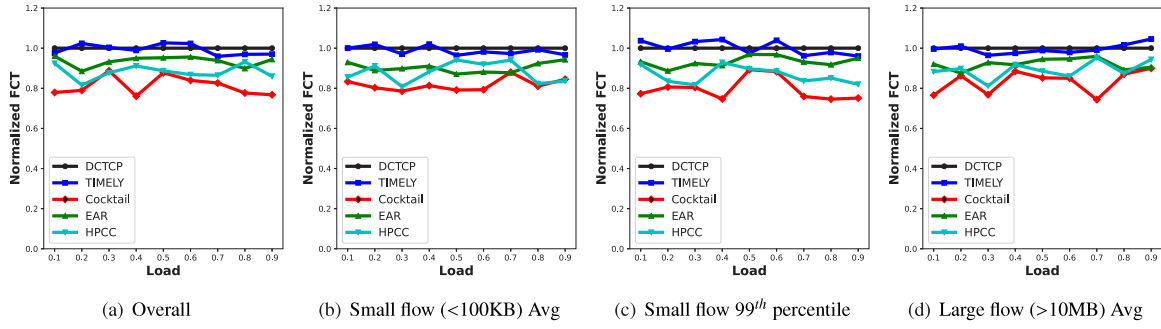


Fig. 9. Cache follower workload: FCT across different flow sizes.

workload and flow sizes. To better visualize the results, we normalize the FCT relative to that of DCTCP.

Under the web search workload: The web search workload mostly contains small and medium-sized flows from a few KB to tens of MB, with more than 95% of total bytes come from the flow smaller than 20 MB [6]. From Fig. 6(a) we observe that *Cocktail* generally achieves the best overall performance, consistent with our experiment results in Section 5.3. For the flows smaller than 100 KB, *Cocktail* significantly reduces the average and the 99th percentile FCT, achieving average reductions of 14.33%, 12.15% and 7.6% FCT compared to DCTCP, TIMELY and EAR respectively.

In a larger flow size group, *Cocktail* greatly outperforms other schemes. It achieves 20.24% lower average FCT compared to DCTCP and 19.34% lower than TIMELY. Furthermore, *Cocktail* obtains up to 16% improvement in FCT when compared to EAR.

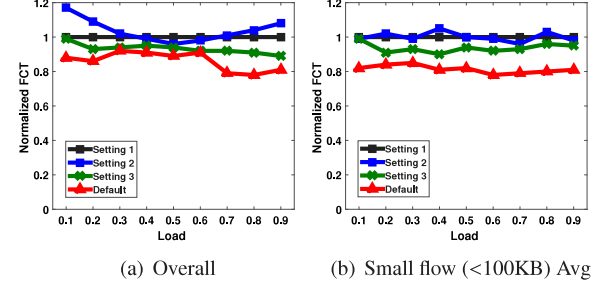
Across all flow size, *Cocktail* can achieve performance comparable to HPCC. Especially for small flow, *Cocktail* demonstrates its effectiveness by reducing FCT by up to 8.3%, and in most cases, *Cocktail* achieves lower FCTs.

Under the data mining workload: The data mining workload comprises both tiny and large-sized flows, ranging from hundreds of bytes to 1 GB. The flow sizes are highly skewed: 80% of the flows are smaller than 10 KB, while 95% of the total bytes come from flows larger than 30 MB [6]. The flow completion times of data mining workload are presented in Fig. 7.

The performance improvement of *Cocktail* is more prominent for small flows in the data mining workload than in the search workload. From Fig. 7(a), *Cocktail* generally achieve the best overall average FCT. Compared to DCTCP, *Cocktail* achieves up to 21% lower FCT. When compared to TIMELY, *Cocktail* outperforms it across all loads for the data mining workload. Moreover, *Cocktail* reduces FCT by up to 16.6% compared to EAR's FCT.

Under the Hadoop workload: The Hadoop workload mostly contains tiny flows. We demonstrate that *Cocktail* also achieves good performance under this workload. *Cocktail* attains comparable overall performance to HPCC while reducing FCT by up to 24.3% and 23.7% compared to DCTCP and TIMELY, respectively. Furthermore, compared to DCTCP, *Cocktail* reduces the average FCT for small flows by up to 20.8% and achieves about 25.9% improvement on FCT compared to TIMELY. When it comes to large flows, *Cocktail* slightly outperforms other schemes.

Under the cache follower workload: In general, *Cocktail* achieves excellent overall performance among all schemes. In particular, *Cocktail* demonstrates comparable performance to HPCC and reduces the average Flow Completion Time (FCT) by about 18.5% compared to TIMELY. The most noticeable performance improvement for small flows is observed in the 99th percentile FCT at 40% load in Fig. 9, where *Cocktail* outperforms DCTCP by up to 24.7% and reduces FCT by approximately 26.9% compared to TIMELY. For large flows, *Cocktail* generally outperforms other three mechanisms and achieves slightly better FCT than HPCC. Further, compared to DCTCP, *Cocktail* reduces the average FCT for large flows by up to 26.4%. *Cocktail* obtains about 24.5% better FCT as TIMELY's.

Fig. 10. FCT with different *Fastbyte*. FCT is normalized to the value achieved by the default setting.

5.5. Impact of *Fastbyte*

In this section, we evaluate *Cocktail*'s robustness to parameter settings through a series of targeted simulations. We analyzed all workloads and average results are shown in Fig. 10. Our objective is to highlight factors influencing threshold values, rather than to fine-tune parameters. The *Fastbyte* parameter is essential to network performance. Generally, the sender transmits *Fastbyte* bytes blindly to cover the round trip time to the receiver and the receiver can return explicit congestion information (e.g., RTT and ECN) to help the sender control future transmissions. Four settings are configured to explore *Cocktail*'s sensitivity to *Fastbyte*, i.e., 5 packets (setting 1), 30 packets (setting 2), 20 packets (setting 3) and 10 packets (default setting). As Fig. 10 illustrates, larger *Fastbyte* can degrade the FCT performance. Since more packets get into the switches, leading to congestion at a single hop. The response from receivers may not arrive at the sender immediately, making congestion inevitable. Conversely, a smaller *Fastbyte* can also hinder performance, as it may underutilize network capacity. Therefore, in our simulations, setting *Fastbyte* to 10 packets achieves the best balance and performance.

6. Related works

To provide acceptable network performance, many schemes combining two signals have been proposed, like EAR [5], ECN# [7], RTT_DCQN [21] and FFC [8]. While these schemes reveal the ineffectiveness of either signal, their mechanism may have limitations on improving performance in multi-bottleneck scenarios.

EAR & ECN#: EAR and ECN# are twins which share similar design principles. So we only discuss EAR below and ECN# encounters similar problems. EAR is a congestion control for datacenter networks, which combines ECN and RTT signals. Specifically, the EAR senders treat ACKs satisfying one of the following conditions as congestion indicators:

$$queue > ecn_threshold(K) \quad (17)$$

$$RTT > base_RTT + delay_threshold(T) \quad (18)$$

where $base_RTT$ is the minimal RTT sample, K and T are the threshold for switch queue and RTT. The congestion window is computed in alignment with the original DCTCP, where senders aggregate congestion indicators over a set window to produce a congestion fraction as the measurement output. The subsequent congestion control intelligence module adjusts the congestion window based on the well-studied control laws of DCTCP, while standard TCP features, including slow start, fast retransmission on triple duplicate ACKs, and fast recovery from packet loss, remain unchanged. A recent method [21] combines delay and CNP feedback for data management, using delay solely to assess congestion upon receiving CNP feedback. However, this hybrid approach does not fully exploit the potential of delay signals.

FFC: FFC is a transport protocol that independently adjusts its congestion window based on two-dimensional congestion notifications from RTT and ECN signals. Specifically, the FFC sender processes each ACK to capture real-time RTT and ECN marks, reducing the congestion window only upon receiving ECN-marked ACKs. Additionally, FFC calculates the window reduction based on the measured RTTs of marked ACKs within the current measurement period. The congestion window is then computed as follows:

$$W_{j+1} = \begin{cases} W_j + 1, & \text{if no ACK is marked} \\ W_j \times \frac{R(E-\hat{q})}{E^2} + \frac{\hat{q}R}{E^2}, & \text{otherwise} \end{cases} \quad (19)$$

where E is the RTT when the queue length is just equal to the marking threshold, $\hat{q}$ is the queuing delay attributed to the part of queue length that exceeds the marking threshold and R represents the minimal RTT.

EAR, in particular, employs two thresholds within a single hop's queue to control queue length at each switch, akin to the mechanism used by DCTCP. Thus, EAR may encounter similarly accumulative large end-to-end latency in multi-bottleneck scenarios, as seen in DCTCP or other ECN-based schemes. Besides, though FFC continually estimates the RTT and ECN fraction, it actually recognizes the ECN as the congestion signal and only uses RTT for shrinking the congestion window when the network is congested. Similarly, RTT_DCQCN only uses delay as rate computation input when the sender receives CNP signals, which confines its sending rate adhering to ECN-based mechanism. These methods fall short of fully leveraging ECN and RTT signals in multi-bottleneck scenarios. On the contrary, *Cocktail* integrates both metrics to optimize network performance, with experiments confirming its effectiveness.

In recent year, Bolt [22], EQCIN [23], SWIFT [24], HPCC [25], Poseidon [26] and ACC [27] introduced new congestion control mechanisms aimed at enhancing transmission performance. However, these schemes rely on in-network telemetry (INT) to accurately obtain congestion information, which poses challenges for large-scale deployment.

Recent research has made significant strides in enhancing TCP performance over high bandwidth-delay product paths, including TCP Wave [28] and BBRv2+ [29]. However, these schemes do not focus on maintaining small queue lengths, which is a critical requirement in datacenter environments [30].

7. Conclusion

In this paper, we present *Cocktail*, a simple yet effective congestion control that uses both ECN and RTT as congestion signals to simultaneously achieve high throughput and low latency simultaneously. Our evaluation results show that *Cocktail* significantly reduces flow completion times in multi-bottleneck scenarios, demonstrating that it is a practical solution that meets all our design objectives.

CRedit authorship contribution statement

Yuxiang Zhang: Writing – review & editing, Writing – original draft, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Lin Cui:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Formal analysis, Conceptualization. **Fung Po Tso:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Xiaolin Lei:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is partially supported by National Natural Science Foundation of China (NSFC) under Grant 62172189; the Innovate UK grants 47198 and 10040850.

Data availability

No data was used for the research described in the article.

References

- [1] M. Alizadeh, A. Greenberg, D.A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, Data center tcp (DCTCP), in: ACM SIGCOMM Computer Communication Review, vol. 40, (4) ACM, 2010, pp. 63–74.
- [2] I. Grigorik, Optimizing the Critical Rendering Path, O'Reilly Velocity Conference, Santa Clara, CA, 2013.
- [3] R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H.C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab, et al., Scaling memcache at facebook, in: Presented As Part of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13), 2013, pp. 385–398.
- [4] I. Cho, K. Jang, D. Han, Credit-scheduled delay-bounded congestion control for datacenters, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication, ACM, 2017, pp. 239–252.
- [5] G. Zeng, W. Bai, G. Chen, K. Chen, D. Han, Y. Zhu, Combining ECN and RTT for Datacenter Transport, in: Proceedings of the First Asia-Pacific Workshop on Networking, ACM, 2017, pp. 36–42.
- [6] C. Lee, C. Park, K. Jang, S.B. Moon, D. Han, Accurate Latency-based Congestion Feedback for Datacenters., in: USENIX Annual Technical Conference, 2015, pp. 403–415.
- [7] J. Zhang, W. Bai, K. Chen, Enabling ECN for datacenter networks with RTT variations, in: Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies, CoNEXT '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 233–245.
- [8] T. Zhang, J. Huang, J. Wang, J. Chen, Y. Pan, G. Min, Designing Fast and Friendly TCP to Fit High Speed Data Center Networks, in: 2018 IEEE 38th International Conference on Distributed Computing Systems, ICDCS, IEEE, 2018, pp. 43–53.
- [9] M. Alizadeh, A. Kabbani, T. Edsall, B. Prabhakar, A. Vahdat, M. Yasuda, Less is More: trading a little bandwidth for ultra-low latency in the data center, in: Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation, USENIX Association, 2012, p. 19.
- [10] M. Alizadeh, S. Yang, M. Sharif, S. Katti, N. McKeown, B. Prabhakar, S. Shenker, pfabric: Minimal near-optimal datacenter transport, in: ACM SIGCOMM Computer Communication Review, vol. 43, (4) ACM, 2013, pp. 435–446.
- [11] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M.H. Yahia, M. Zhang, Congestion control for large-scale RDMA deployments, in: ACM SIGCOMM Computer Communication Review, vol. 45, (4) ACM, 2015, pp. 523–536.
- [12] R. Mittal, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, D. Zats, et al., TIMELY: RTT-based Congestion Control for the Datacenter, in: ACM SIGCOMM Computer Communication Review, vol. 45, (4) ACM, 2015, pp. 537–550.
- [13] D.-M. Chiu, R. Jain, Analysis of the increase and decrease algorithms for congestion avoidance in computer networks, Comput. Netw. ISDN syst. 17 (1) (1989) 1–14.

- [14] Y. Zhu, M. Ghobadi, V. Misra, J. Padhye, ECN or Delay: Lessons Learnt from Analysis of DCQCN and TIMELY, in: Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies, ACM, 2016, pp. 313–327.
- [15] B. Montazeri, Y. Li, M. Alizadeh, J. Ousterhout, Homa: A receiver-driven low-latency transport protocol using network priorities, 2018, arXiv preprint [arXiv: 1803.09615](https://arxiv.org/abs/1803.09615).
- [16] J. Ye, K.-C. Leung, V.O. Li, S.H. Low, Combating bufferbloat in multi-bottleneck networks: Equilibrium, stability, and algorithms, in: IEEE INFOCOM 2018-IEEE Conference on Computer Communications, IEEE, 2018, pp. 648–656.
- [17] W. Bai, K. Chen, L. Chen, C. Kim, H. Wu, Enabling ECN over generic packet scheduling, in: Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies, ACM, 2016, pp. 191–204.
- [18] W. Bai, L. Chen, K. Chen, H. Wu, Enabling ECN in Multi-Service Multi-Queue Data Centers., in: NSDI, 2016, pp. 537–549.
- [19] A. Greenberg, J.R. Hamilton, N. Jain, S. Kandula, C. Kim, P. Lahiri, D.A. Maltz, P. Patel, S. Sengupta, VL2: a scalable and flexible data center network, in: ACM SIGCOMM Computer Communication Review, vol. 39, (4) ACM, 2009, pp. 51–62.
- [20] A. Roy, H. Zeng, J. Bagga, G. Porter, A.C. Snoeren, Inside the social network's (datacenter) network, in: ACM SIGCOMM Computer Communication Review, vol. 45, (4) ACM, 2015, pp. 123–137.
- [21] Y. Wang, M. Lan, T. Zhao, Z. Gao, M. Xie, Combining RTT and ECN for RoCEv2 Protocol, in: HPCCT & BDAI '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 158–164, <http://dx.doi.org/10.1145/3409501.3409509>.
- [22] S. Arslan, Y. Li, G. Kumar, N. Dukkkipati, Bolt: Sub-RTT congestion control for Ultra-Low latency, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, Boston, MA, 2023, pp. 219–236, URL <https://www.usenix.org/conference/nsdi23/presentation/arslan>.
- [23] P. Dong, X. Lu, T. Huang, L. Chen, Y. Yang, L. Zhang, Predictive queue-based rate control for low latency in lossless data center networks, IEEE Trans. Netw. Serv. Manag. (2024).
- [24] G. Kumar, N. Dukkkipati, K. Jang, H.M. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, et al., Swift: Delay is simple and effective for congestion control in the datacenter, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, 2020, pp. 514–528.
- [25] Y. Li, R. Miao, H.H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, et al., HPCC: High precision congestion control, in: Proceedings of the ACM Special Interest Group on Data Communication, 2019, pp. 44–58.
- [26] W. Wang, M. Moshref, Y. Li, G. Kumar, T.S.E. Ng, N. Cardwell, N. Dukkkipati, Poseidon: Efficient, robust, and practical datacenter CC via deployable INT, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, Boston, MA, 2023, pp. 255–274, URL <https://www.usenix.org/conference/nsdi23/presentation/wang-weitao>.
- [27] Y. Zhang, Q. Meng, C. Hu, F. Ren, Revisiting congestion control for lossless ethernet, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), 2024, pp. 131–148.
- [28] A. Abdelsalam, M. Luglio, C. Roseti, F. Zampognaro, TCP wave: a new reliable transport approach for future internet, Comput. Netw. 112 (2017) 122–143.
- [29] F. Yang, Q. Wu, Z. Li, Y. Liu, G. Pau, G. Xie, BBRv2+: Towards balancing aggressiveness and fairness with delay-based bandwidth probing, Comput. Netw. 206 (2022) 108789.

- [30] A. Abdelsalam, M. Luglio, M. Quadrini, C. Roseti, F. Zampognaro, QUIC-proxy based architecture for satellite communication to enhance a 5G scenario, in: 2019 International Symposium on Networks, Computers and Communications, ISNCC, IEEE, 2019, pp. 1–6.



Yuxiang Zhang received his the BEng, and MEng degree from Jinan University, China, in 2015 and 2018 respectively. He has broad interests in networking systems, with focuses on the area of datacenter networking and network optimization.



Lin Cui is currently with the Department of Computer Science at Jinan University, Guangzhou, China. He received the Ph.D. degree from City University of Hong Kong in 2013. He has broad interests in networking systems, with focuses on the following topics: programmable networks, data center networking, software defined networking (SDN) and so on.



Fung Po Tso received his BEng, MPhil and Ph.D. degrees from City University of Hong Kong in 2006, 2007 and 2011 respectively. He is currently a senior lecturer in the Department of Computer Science at the Loughborough University. Prior to joining Loughborough, he worked as SICS Next Generation Internet Fellow at the School of Computing Science, University of Glasgow during 2011–2014 and lecturer in Liverpool John Moores University during 2014–2017. His research interests include: network policy management, cloud data centre resource management, data centre networking, software defined networking (SDN), and distributed systems.



Xiaolin Lei is currently a faculty member in the Department of Computer Science at Jinan University. He was graduated from Xidian University. His primary research interests include artificial intelligence and networking systems.